ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

TOÁN ỨNG DỤNG VÀ THỐNG KÊ (MTH00051)

BÁO CÁO ĐỒ ÁN 1

Ma Trận và Cơ Sở của Tính Toán Khoa Học

Môn học:	Toán ứng dụng và thống kê	
Giảng viên:	ThS. Võ Nam Thục Đoan	
	ThS. Lê Nhựt Nam	
	TS. Đỗ Đức Hào	
	ThS. Nguyễn Hữu Toàn	
Sinh viên thực hiện:	Lê Hoàng Hiếu	(MSSV: 24120048)
	Lê Nguyễn Gia Huy	(MSSV: 24120061)
	Trần Nguyễn Anh Khoa	(MSSV: 24120075)
	Nguyễn Hoàng Nam	(MSSV: 24120097)
	Nguyễn Anh Sang	(MSSV: 24120131)
Lớp:	24CTT2	

Thành phố Hồ Chí Minh, 04/2026

Mục lục

Danh sách thành viên và Phân công công việc 4

1 Phần 1: Phép khử Gauss và Các Ứng Dụng 5

1.1 Cơ sở lý thuyết 5

1.1.1 Phép khử Gauss 5

1.1.2 Khử Gauss có chọn phần tử chốt (Partial Pivoting) 6

1.1.3 Tính Định Thức qua Khử Gauss 6

1.1.4 Ma Trận Nghịch Đảo 7

1.1.5 Hạng Ma Trận và Cơ Sở 7

1.2 Mô tả thuật toán và Cài đặt 8

1.2.1 Source code hàm khởi tạo và định nghĩa ma trận: 8

1.2.2 Thuật toán khử Gauss bằng Partial Pivoting 9

1.2.3 Giải hệ tam giác thông qua khử Gauss 11

1.2.4 Thuật toán tính định thức qua khử Gauss 14

1.2.5 Thuật toán nghịch đảo ma trận qua khử Gauss 15

1.2.6 Thuật toán tìm hạng ma trận và cơ sở 18

1.2.7 Kiểm thử tính đúng đắn của hàm bằng Numpy, Scipy 20

2 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa 21

2.1 Lý thuyết và Phương pháp phân rã SVD 21

2.1.1 Cơ sở lý thuyết của thuật toán phân rã SVD 21

2.1.2 Mối liên hệ với Bài toán Trị riêng và cách xác định: 22

2.2 Cơ sở lý thuyết về Chéo hóa ma trận 22

2.2.1 Định nghĩa và Điều kiện 22

2.2.2 Trường hợp Ma trận Đối xứng 23

2.3 Cài đặt thuật toán SVD và Chéo hóa ma trận 23

2.3.1 Thuật toán phân rã SVD 23

2.3.2 Thuật toán chéo hoá ma trận 29

2.3.3 Kiểm thử hàm phẫn rã SVD và chéo hoá ma trận 32

2.4 Mô tả kịch bản trực quan hóa (Manim) 32

2.4.1 Trình bày lý thuyết và mô phỏng phân rã SVD 32

2.4.2 Trình bày lý thuyết và mô phỏng Chéo hóa ma trận 34

2.4.3 Mối liên hệ đại số và Ứng dụng nén ảnh 36

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng 38

3.1 Cơ sở lý thuyết về sai số và tính ổn định số 38

3.1.1 Chuẩn và Số điều kiện 38

3.1.2 Sai số tương đối 39

3.1.3 So sánh các phương pháp được thực hiện 39

3.1.4 Phương pháp lặp Gauss-Seidel 39

3.2 Mô tả thuật toán và cài đặt 40

3.2.1 Các bước thực hiện thuật toán Gauss-Seidel 40

3.2.2 Cài đặt thuật toán Gauss-Seidel 41

3.3 Mô tả quá trình kiểm nghiệm 43

3.3.1 Phân loại các trường hợp kiểm thử 43

3.3.2 Các chỉ số đo đạc 43

3.3.3 Quá trình kiểm thử 44

3.4 Phân tích tính ổn định số 44

3.4.1 Ma trận Hilbert 44

3.4.2 Ma trận đối xứng xác định dương (SPD) 45

3.4.3 Mô tả phương pháp ổn định số học 46

4 Phần 4: Kết luận 48

4.1 Những gì đạt được thông qua đồ án 48

4.2 Những khó khăn gặp phải 48

4.3 Bài học rút ra 49

Giới thiệu

Đại số tuyến tính số (Numerical Linear Algebra) đóng vai trò là "trái tim" của hầu hết các hệ thống tính toán hiện đại, từ xử lý đồ họa máy tính, tối ưu hóa hệ thống cho đến các mô hình học máy (Machine Learning) phức tạp. Việc hiểu rõ cách máy tính thực hiện các phép toán ma trận không chỉ dừng lại ở mặt lý thuyết, mà còn quyết định trực tiếp đến hiệu năng và tính chính xác của các phần mềm thực tế.

Nhằm củng cố kiến thức nền tảng và rèn luyện kỹ năng cài đặt thuật toán, đồ án này được thực hiện với trọng tâm hướng đến việc mổ xẻ, lập trình và trực quan hóa các kỹ thuật cốt lõi của đại số tuyến tính trên máy tính. Thay vì phụ thuộc vào các "hộp đen" (*black-box*) từ các thư viện có sẵn, đồ án đặt ra thách thức xây dựng hệ thống tính toán từ con số không (*from scratch*), qua đó hiểu sâu sắc hơn về bản chất của các phép biến đổi không gian và những rủi ro về sai số học trong quá trình tính toán.

Giới hạn môi trường triển khai

Để đảm bảo tính minh bạch và đánh giá đúng năng lực tự cài đặt thuật toán, đồ án tuân thủ nghiêm ngặt các quy định về công cụ và môi trường phát triển:

- **Ngôn ngữ cốt lõi:** Toàn bộ thuật toán được cài đặt hoàn toàn bằng **Python** (phiên bản 3.10 trở lên). Không sử dụng bất kỳ thư viện hỗ trợ tính toán ma trận nào cho phần logic chính.
- **Công cụ đối chiếu và xử lý phụ trợ:** Các thư viện chuyên dụng như *NumPy, SciPy, SymPy* chỉ được phép sử dụng với vai trò làm tham chiếu (kiểm chứng độ chính xác của kết quả do thuật toán tự viết tạo ra). Các module *math, fractions* được sử dụng hạn chế trong việc định dạng hiển thị và xử lý sai số học.
- **Công cụ mô phỏng và báo cáo:** Sử dụng **Manim Community** (v.20.1) để xây dựng các video trực quan hóa đồ họa toán học sinh động. Kết quả thực nghiệm, so sánh hiệu năng được biểu diễn qua biểu đồ bằng **Matplotlib** và trình bày một cách có hệ thống trên môi trường **Jupyter Notebook**.

Cấu trúc của báo cáo

Báo cáo này được chia thành các phần chính sau:

- **Phần 1: Phép khử Gauss và các ứng dụng.**
- **Phần 2: Phân rã ma trận và trực quan hóa với Manim.**
- **Phần 3: Giải hệ phương trình và phân tích hiệu năng.**
- **Phần 4: Kết luận.**

Kho báo cáo Github: https://github.com/KhoaTapCode2006/do_an_tudtk

Danh sách thành viên và Phân công công việc

STT	MSSV	Họ và Tên	Nhiệm vụ phân công	Mức độ hoàn thành
1	24120048	Lê Hoàng Hiếu	Cài đặt hàm nghịch đảo ma trận, viết báo cáo hàm và mô tả thuật toán	100%
2	24120131	Nguyễn Anh Sang	Cài đặt hàmn tìm hạng, cơ sở dòng, cột, nghiệm, viết báo cáo hàm và mô tả thuật toán	100%
3	24120075	Trần Nguyễn Anh Khoa	Cài đặt hàm khử Gauss, mô tả thuật toán, kiểm thử các hàm bằng Jupyter Notebook, fix code, hoàn thiện báo cáo phần 1	100%
4	24120061	Lê Nguyễn Gia Huy	Cài đặt hàm tính định thức, viết báo cáo và mô tả thuật toán	100%
5	24120097	Nguyễn Hoàng Nam	Cài đặt hàm giải hệ tam giác, viết báo cáo hàm và mô tả thuật toán	100%

Bảng 1: BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM - PHẦN 1

STT	MSSV	Họ và Tên	Nhiệm vụ phân công	Mức độ hoàn thành
1	24120048	Lê Hoàng Hiếu	Lồng tiếng video Manim, bổ sung mô phỏng chéo hoá ma trận	100%
2	24120131	Nguyễn Anh Sang	Làm video Manim (Cắt ghép, viết code, chỉnh sửa code), viết báo cáo ý tưởng video Manim, mô phỏng ứng dụng nén ảnh bằng SVD trên Manim	100%
3	24120075	Trần Nguyễn Anh Khoa	Cài đặt hàm chéo hoá ma trận, kiểm thử hàm chéo hoá, phân rã, viết báo cáo hàm và mô tả thuật toán	100%
4	24120061	Lê Nguyễn Gia Huy	Cài đặt hàm phân rã ma trận (Phương pháp phân rã SVD), viết báo cáo hàm và mô tả thuật toán	100%
5	24120097	Nguyễn Hoàng Nam	Lồng tiếng video Manim	100%

Bảng 2: BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM - PHẦN 2

STT	MSSV	Họ và Tên	Nhiệm vụ phân công	Mức độ hoàn thành
1	24120048	Lê Hoàng Hiếu	Tạo bộ dữ liệu kiểm thử, đo đạc, kiểm thử 3 phương pháp khử dựa trên thời gian, sai số, số bước	100%
2	24120131	Nguyễn Anh Sang	Vẽ biểu đồ thời gian, độ hội tụ và đồ thị log-log. Phân tích tính ổn định số cho 2 loại ma trận Hilbert và SPD	100%
3	24120075	Trần Nguyễn Anh Khoa	Viết báo cáo cơ sở lý thuyết tính ổn định số, mô tả thuật toán Gauss Seidel, mô tả quá trình kiểm thử, làm README.md, hoàn thiện report	100%
4	24120061	Lê Nguyễn Gia Huy	Suy luận và đánh giá độ phù hợp và tình huống khi sử dụng các phương pháp khử dựa trên thời gian, sai số	100%
5	24120097	Nguyễn Hoàng Nam	Cài đặt hàm khử cho 3 phương pháp, kiểm tra điều kiện hội tụ cho Gauss Seidel	100%

Bảng 3: BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM - PHẦN 3

1 Phần 1: Phép khử Gauss và Các Ứng Dụng

1.1 Cơ sở lý thuyết

1.1.1 Phép khử Gauss

Định nghĩa Phép khử Gauss:

Phép khử Gauss (Gaussian Elimination) là một thuật toán trong đại số tuyến tính nhằm biến đổi một ma trận về dạng bậc thang (Row Echelon Form - REF). Đối với hệ phương trình tuyến tính $Ax = b$, phương pháp này sử dụng các phép biến đổi hàng sơ cấp để chuyển ma trận mở rộng $[A|b]$ thành ma trận tam giác trên, từ đó giải ngược (chuyển vế) để tìm nghiệm.

Công thức khử:

Giả sử tại bước thứ k của quá trình khử, ta cần loại bỏ các phần tử nằm dưới đường chéo chính của cột k. Với mỗi hàng i ($i > k$), hệ số nhân được tính bằng:

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} \tag{1}$$

Phép biến đổi cập nhật hàng i (ký hiệu là R_i) được thực hiện theo công thức:

$$R_i \leftarrow R_i - m_{ik} R_k$$

Sau phép biến đổi này, phần tử tại vị trí (i, j) của ma trận ở bước $k+1$ sẽ được cập nhật thành:

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} a_{kj}^{(k)} \quad \text{với } j \geq k \tag{2}$$

1.1.2 Khử Gauss có chọn phần tử chốt (Partial Pivoting)

Định nghĩa Phép khử chọn phần tử chốt:

Thuật toán khử Gauss sẽ thất bại nếu phần tử trên đường chéo chính $a_{kk}^{(k)} = 0$ hoặc gây ra sai số làm tròn rất lớn nếu $a_{kk}^{(k)}$ quá gần 0. Kỹ thuật Partial Pivoting được áp dụng để khắc phục và giảm thiểu sai số làm tròn.

Công thức khử phần tử chốt

Trước khi thực hiện bước khử thứ k, ta tìm chỉ số hàng p (với $p \geq k$) sao cho giá trị tuyệt đối của phần tử thuộc cột k là lớn nhất:

$$|a_{pk}^{(k)}| = \max_{i=k}^{n} |a_{ik}^{(k)}| \tag{3}$$

Sau đó, ta hoán đổi dòng k và dòng p trước khi thực hiện khử

1.1.3 Tính Định Thức qua Khử Gauss

Định nghĩa Định thức:

Định thức (determinant), ký hiệu là $\det(A)$ hoặc $|A|$, là một giá trị vô hướng đặc trưng cho ma trận vuông $A \in \mathbb{R}^{n \times n}$. Định thức cung cấp thông tin quan trọng về tính khả nghịch của ma trận: ma trận A khả nghịch khi và chỉ khi $\det(A) \neq 0$. Trong hình học, trị tuyệt đối của định thức biểu diễn tỷ lệ thay đổi diện tích hoặc thể tích của các đa bào qua phép biến đổi tuyến tính.

Công thức tính Định thức qua khử Gauss:

Dựa trên các tính chất của định thức đối với phép biến đổi hàng sơ cấp, ta có thể tính định thức bằng cách đưa ma trận về dạng tam giác trên U thông qua s lần hoán vị hàng. Khi đó, định thức được tính bằng công thức:

$$\det(A) = (-1)^s \cdot \prod_{i=1}^{n} u_{ii} \tag{4}$$

Trong đó:

- u_{ii} là các phần tử nằm trên đường chéo chính của ma trận tam giác trên U.

- s là số lần thực hiện hoán vị hàng (Partial Pivoting) trong suốt quá trình khử.
- $(-1)^s$ là hệ số điều chỉnh dấu do tính chất hoán vị hàng làm đổi dấu định thức.

1.1.4 Ma Trận Nghịch Đảo

Định nghĩa:

Cho ma trận vuông $\mathbf{A}$ cấp n. Ma trận $\mathbf{A}^{-1}$ được gọi là nghịch đảo của $\mathbf{A}$ nếu:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n \tag{5}$$

Công thức tổng quát: Điều kiện tồn tại là $\det(\mathbf{A}) \neq 0$. Khi đó:

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\text{adj}(\mathbf{A}) \tag{6}$$

Trong đó $\text{adj}(\mathbf{A})$ là ma trận phụ hợp, chứa các phần bù đại số $C_{ij} = (-1)^{i+j}M_{ij}$ đã được chuyển vị.

Phương pháp Gauss-Jordan: Biến đổi ma trận mở rộng bằng các phép biến đổi hàng sơ cấp:

$$[\mathbf{A}|\mathbf{I}] \sim [\mathbf{I}|\mathbf{A}^{-1}] \tag{7}$$

1.1.5 Hạng Ma Trận và Cơ Sở

Định nghĩa hạng ma trận (Rank):

Hạng của ma trận $A \in \mathbb{R}^{m\times n}$, ký hiệu là $\text{rank}(A)$, là số lượng tối đa các hàng (hoặc cột) độc lập tuyến tính của ma trận đó. Qua quá trình khử Gauss, hạng của ma trận chính là số lượng các phần tử chốt (pivot) khác 0 trên ma trận bậc thang thu được.

Các không gian cơ bản và Cơ sở (Basis):

Một cơ sở của không gian vector là một tập hợp các vector độc lập tuyến tính và có khả năng sinh ra toàn bộ không gian đó. Từ ma trận A, ta có thể xác định 3 không gian cốt lõi:

- **Không gian dòng (Row Space):** Là không gian sinh bởi các vector hàng của A. Khi đưa A về dạng bậc thang (REF) thông qua phép khử Gauss, các hàng khác 0 của ma trận bậc thang chính là một cơ sở của không gian dòng.
- **Không gian cột (Column Space):** Là không gian sinh bởi các vector cột của A. Để tìm cơ sở, ta xác định vị trí các cột chứa phần tử chốt trong ma trận bậc thang, sau đó lấy các

cột tương ứng từ ma trận A **ban đầu**.

- **Không gian nghiệm (Null Space):** Là tập hợp mọi vector x thỏa mãn phương trình thuần nhất $Ax = 0$. Theo định lý Hạng - Độ lệch (Rank-Nullity Theorem), số chiều của không gian nghiệm bằng $n - \text{rank}(A)$. Cơ sở của không gian này được tìm bằng cách biểu diễn các biến cơ sở theo các biến tự do (quá trình thế ngược).

1.2 Mô tả thuật toán và Cài đặt

1.2.1 Source code hàm khởi tạo và định nghĩa ma trận:

Listing 1: Hàm khởi tạo và định nghĩa ma trận - matrix.py

```python
class Matrix:
    Zero = 1e-10

    def __init__(self, matrix, b=None):
        self.A = [list(map(float, row)) for row in matrix]
        self.n = len(self.A)
        self.m = len(self.A[0])
        self.b = [float(x) for x in b] if b is not None else None

    @staticmethod
    def swapRows(mat, i, j):
        mat[i], mat[j] = mat[j], mat[i]

    @staticmethod
    def copyMatrix(mat):
        return [row[:] for row in mat]

    def display(self, mat=None, title="Ma trn"):
        print(f"--- {title} ---")
        target = mat if mat is not None else self.A
        for row in target:
            print([round(x, 4) for x in row])
```

1.2.2 Thuật toán khử Gauss bằng Partial Pivoting

Đầu vào:

- Ma trận hệ số $A \in \mathbb{R}^{n \times n}$
- Vector kết quả $b \in \mathbb{R}^n$.

Đầu ra:

- Ma trận bậc thang tam giác trên $[A|b]$
- Vector b sau khi đã khử
- Số lần hoán đổi dòng

Mô tả các bước:

Bước 1: Khởi tạo ma trận M = $[A|b]$

Bước 2: Ở bước thứ k, tìm hàng thứ p $(p \geq k)$ sao cho $|Mpk| = max|Mik| : k \leq i \leq n$

Bước 3: Nếu $|Mpk| = 0$, báo lỗi không tồn tại pivot tại cột k. Nếu $|Mpk| < \lambda$ với $\lambda = 1e{-}10$

Bước 4: Nếu hàng $p \neq k$, hoán đổi hàng k và p

Bước 5: Tính hệ số nhân: $m = \frac{M_{ik}}{M_{kk}}$ và cập nhật $M_i \leftarrow M_i - m \times M_k$

Listing 2: Hàm khử Gauss bằng Partial Pivoting - $gaussian_eliminate.py$

```python
import matrix as mt

def gaussian_eliminate(A, b=None):
    if not A or not A[0]:
        raise ValueError("Ma tran A khong duoc rong.")

    n = len(A)
    m = len(A[0])

    if b is not None:
        if len(b) != n:
            raise ValueError("Kich thuoc vector b phai khop voi so hang cua ma tran A.")
        b_new = list(b) # Tao ban sao de tranh thay doi du lieu goc
    else:
        b_new = None
```

```python

    # Khoi tao ma tran moi
    newMatrix = mt.Matrix.copyMatrix(A)
    swaps = 0
    row = 0

    for col in range(m):
        if row >= n:
            break

        # Tim phan tu chot lon nhat (dong co tri tuyet lon nhat)
        maxRow = row
        for i in range(row + 1, n):
            if abs(newMatrix[i][col]) > abs(newMatrix[maxRow][col]):
                maxRow = i

        # Neu phan tu chot lon nhat van gan bang 0, cot nay khong the khu tiep
        if abs(newMatrix[maxRow][col]) < mt.Matrix.Zero:
            continue

        # Neu dong duoc chon khac voi dong dang chay thi hoan doi 2 dong do
        if maxRow != row:
            mt.Matrix.swapRows(newMatrix, maxRow, row)
            if b_new is not None:
                b_new[row], b_new[maxRow] = b_new[maxRow], b_new[row]
            swaps += 1

        # Khu Gauss
        for i in range(row + 1, n):
            # Tinh he so nhan
            factor = newMatrix[i][col] / newMatrix[row][col]

            # Khu tren ma tran A
            for j in range(col, m):
                newMatrix[i][j] -= factor * newMatrix[row][j]

```

```
        # Khu tren vector b
        if b_new is not None:
            b_new[i] -= factor * b_new[row]

    row += 1

return newMatrix, b_new, swaps
```

1.2.3 Giải hệ tam giác thông qua khử Gauss

Thuật toán được thiết kế để giải hệ phương trình tuyến tính dạng tam giác trên $Ux = c$, trong đó U là ma trận ở dạng bậc thang thu được sau khi thực hiện khử Gauss.

Đầu vào:

- Ma trận $U \in \mathbb{R}^{n \times n}$
- Vector c sau khi thông qua phép khử Gauss

Đầu ra:

- Nghiệm duy nhất: trả về vector chứa các nghiệm của hệ
- Vô nghiệm: thông báo hệ vô nghiệm và trả về lại ma trận U và vector c
- Vô số nghiệm: đưa ra công thức nghiệm tổng quát của hệ

Mô tả các bước:

Bước 1: Xác định vị trí các phần tử chốt trên từng dòng.

Bước 2: Duyệt qua từng dòng của ma trận, nếu tồn tại dòng có dạng

$$[0 \quad 0 \quad \dots \quad 0 \quad | \quad k] \text{ với } k \neq 0$$

thì hệ vô nghiệm, trả về kết quả và kết thúc thuật toán.

Bước 3: Phân loại các biến cơ sở $pivot_cols_set$ và các biến tự do $free_vars$

Bước 4: Tiến hành thế ngược, có hai trường hợp xảy ra:

- Nếu $free_vars$ rỗng, hệ có nghiệm duy nhất và được xác định thông qua công thức:

$$x_i = \frac{1}{M_{ii}} \left(M_{i,m+1} - \sum_{j=i+1}^{n} M_{ij} x_j \right), \quad i = n, n-1, \dots, 1.$$

- Nếu $free_vars$ không rỗng, hệ có vô số nghiệm. Khi này ta sẽ xây dựng công thức nghiệm tổng quát thông qua nghiệm riêng $x_particular$ (gán cho các biến tự do bằng 0) và cơ sở không gian nghiệm $Ux = 0$ (gán biến tự do thứ $k = 1$ và các biến tự do còn lại bằng 0)

Listing 3: Hàm giải hệ tam giác trên Ux = c - $back_substitution.py$

```python
import matrix as mt
import gaussian_eliminate as ge

def back_substitution(U, c):
    """
        Giai he phuong trinh tam giac tren Ux = c.
        Xu ly 3 truong hop: nghiem duy nhat, vo nghiem, vo so nghiem.
    """
    # Xac dinh so dong va so cot cua ma tran bac thang U
    n_rows = len(U)
    n_cols = len(U[0])

    # Xac dinh vi tri cac phan tu chot tren tung dong
    pivot_col = []
    for i in range(n_rows):
        pc = -1
        for j in range(n_cols):
            # Neu phat hien pivot, danh dau vi tri cot chua no
            if abs(U[i][j]) > mt.Matrix.Zero:
                pc = j
                break
        pivot_col.append(pc)

    # Kiem tra he phuong trinh co vo nghiem khong
    for i in range(n_rows):
        # Neu ton tai dong co dang [0 0 0 ... 0 | k] voi k != 0, he vo nghiem
        if pivot_col[i] == -1 and abs(c[i]) > mt.Matrix.Zero:
            return 0, None

    # Luu cac bien co so tuong ung voi cac cot co phan tu chot
```

```python
pivot_cols_set = {pc for pc in pivot_col if pc != -1}
# Luu cac bien tu do tuong ung voi cac cot khong co phan tu chot
free_vars = [j for j in range(n_cols) if j not in pivot_cols_set]

# Thuat toan the nguoc
def solve_system(c, free_values):
    # Khoi tao vector nghiem x voi kich thuoc bang so cot
    x = [0.0] * n_cols

    # Gan gia tri an tu do
    for k, fv in enumerate(free_vars):
        x[fv] = free_values[k]

    # Duyet nguoc vi gia tri an o dong duoi se giup giai an o dong tren
    for i in range(n_rows - 1, -1, -1):
        pc = pivot_col[i]

        # Neu dong toan so 0, bo qua
        if pc == -1:
            continue

        # x_pc = (c_i - tong cac phan tu da biet) / he so tai pivot
        sigma = sum(U[i][j] * x[j] for j in range(pc + 1, n_cols))
        x[pc] = (c[i] - sigma) / U[i][pc]

    return x

# Nghiem duy nhat
if not free_vars:
    return 1, solve_system(c, [])

# Vo so nghiem
if free_vars:
    # Gan tat ca bien tu do = 0
    particular_sol = solve_system(c, [0.0] * len(free_vars))

```

```python
    # He nghiem co so (Null space basis)
    basis = []
    for i in range(len(free_vars)):
        # Tao vector tri tu do: bien thu i la 1.0, con lai la 0.0
        free_values = [0.0] * len(free_vars)
        free_values[i] = 1.0

        # Giai Ux = 0
        zero_vec = [0.0] * n_rows
        vec_basis = solve_system(zero_vec, free_values)
        basis.append(vec_basis)

    return -1, (particular_sol, basis)
```

1.2.4 Thuật toán tính định thức qua khử Gauss

Thuật toán tính định thức dựa trên việc tận dụng kết quả của quá trình khử Gauss với chọn phần tử chốt (Partial Pivoting) để đưa ma trận về dạng tam giác trên, giúp tối ưu độ phức tạp tính toán.

Đầu vào:

- Ma trận vuông $A \in \mathbb{R}^{n \times n}$.

Đầu ra:

- Giá trị định thức $\det(A)$.

Mô tả các bước:

Bước 1: Kiểm tra tính hợp lệ của ma trận đầu vào. Nếu số hàng khác số cột ($n \neq m$), báo lỗi vì định thức chỉ xác định cho ma trận vuông.

Bước 2: Thực hiện thuật toán khử Gauss với Partial Pivoting cho ma trận A. Thuật toán trả về:

- Ma trận bậc thang tam giác trên U.
- Biến `swaps`: Tổng số lần hoán đổi hàng đã thực hiện trong quá trình khử.

Bước 3: Khởi tạo biến tích `det = 1.0`. Duyệt qua đường chéo chính của ma trận U, thực hiện nhân dồn các phần tử:

$$\det = \prod_{i=0}^{n-1} U[i][i] \tag{8}$$

Bước 4: Điều chỉnh dấu của định thức dựa trên số lần hoán vị hàng (`swaps`). Nếu `swaps` là số lẻ, ta đổi dấu định thức:

$$\det = (-1)^{\text{swaps}} \times \det \tag{9}$$

Bước 5: Trả về giá trị `det` cuối cùng.

Cài đặt thuật toán: Dưới đây là đoạn mã nguồn minh họa việc hiện thực hàm tính định thức:

Listing 4: Hàm tính định thức của ma trận vuông - *determinant.py*

```python
import gaussian_eliminate as ge
import matrix
def determinant(A):
    n = len(A)
    # Goi ham khu Gauss (truyen None cho vector b vi chi can ma tran U)
    newMatrix, _, swaps = ge.gaussian_eliminate(A, None)

    # Tinh tich cac phan tu tren duong cheo chinh
    det = 1.0
    for i in range(n):
        det *= newMatrix[i][i]

    # Doi dau dinh thuc neu so lan hoan doi hang la so le
    if swaps % 2 != 0:
        det = -det

    return det
```

1.2.5 Thuật toán nghịch đảo ma trận qua khử Gauss

Thuật toán tìm ma trận nghịch đảo A^{-1} bằng phương pháp Gauss-Jordan thực hiện các phép biến đổi hàng sơ cấp trên ma trận mở rộng $[A|I]$. Mục tiêu của thuật toán là biến đổi khối

ma trận A (bên trái) thành ma trận đơn vị I. Khi đó, khối ma trận đơn vị I (bên phải) sẽ tự động chuyển thành ma trận nghịch đảo A^{-1}. Khác với phương pháp khử Gauss thông thường, Gauss-Jordan thực hiện triệt tiêu các phần tử cả ở phía trên và phía dưới đường chéo chính.

Đầu vào:

- Ma trận vuông $A \in \mathbb{R}^{n \times n}$.

Đầu ra:

- Ma trận nghịch đảo A^{-1} hoặc thông báo lỗi.

Mô tả các bước:

Bước 1: Khởi tạo ma trận mở rộng. Kiểm tra tính hợp lệ của ma trận (phải là ma trận vuông và không rỗng). Tiến hành ghép ma trận đơn vị I_n vào bên phải ma trận A để tạo thành ma trận mở rộng $[A|I]$ kích thước $n \times 2n$.

Bước 2: Tìm Pivot và xử lý hoán vị hàng (Partial Pivoting). Gọi hàm $gaussian_eliminate()$ để tiến hành khử Gauss theo Partial Pivoting

Bước 3: Chuẩn hóa hàng chứa Pivot. Chia toàn bộ hàng col cho giá trị pivot để đưa phần tử tại vị trí đường chéo (col, col) về giá trị 1.0.

Bước 4: Khử Gauss-Jordan (Toàn phần). Duyệt qua tất cả các hàng i khác hàng col. Sử dụng hàng col đã chuẩn hóa để triệt tiêu toàn bộ các phần tử tại cột col ở các hàng khác về 0:

$$R_i = R_i - (A_{i,col} \cdot R_{col})$$

Sau khi kết thúc các vòng lặp, vế trái của ma trận mở rộng sẽ trở thành ma trận đơn vị.

Bước 5: Trích xuất kết quả. Cắt lấy n cột cuối cùng (nửa bên phải) của ma trận mở rộng để thu được ma trận nghịch đảo A^{-1}.

Cài đặt thuật toán:

Listing 5: Ham tinh ma tran nghich dao bang Gauss-Jordan - *inverse.py*

```python
def inverse(self, A):
    """ Tim ma tran nghich dao bang phuong phap Gauss-Jordan tren ma tran mo rong [A |
        I] """
    n = len(A)
    if n == 0:
        raise ValueError("Matrix is empty")
```

```python
    if n != len(A[0]):
        raise ValueError("The matrix is not a square one")

    # tao ma tran mo rong [A|I]
    aug = [row[:] + [1.0 if i == j else 0.0 for j in range(n)] for i, row in
        enumerate(A)]

    for col in range(n):
        # tim hang chua pivot lon nhat
        max_row = col
        for i in range(col + 1, n):
            if abs(aug[i][col]) > abs(aug[max_row][col]):
                max_row = i

        # kiem tra ma tran suy bien
        if abs(aug[max_row][col]) <= mt.Matrix.Zero:
            raise ValueError("Can't find inverse of singular matrix")

        # hoan doi hang hien tai voi hang co pivot lon nhat
        aug[col], aug[max_row] = aug[max_row], aug[col]

        # chia ca hang cho phan tu duong cheo de no bien thanh so 1
        pivot = aug[col][col]
        for j in range(col, 2 * n):
            aug[col][j] /= pivot

        # bien tat ca cac so KHAC (tren va duoi) thanh so 0
        for i in range(n):
            if i != col:
                factor = aug[i][col]
                for j in range(col, 2 * n):
                    aug[i][j] -= factor * aug[col][j]

    # lay nua ben phai, tuc ma tran nghich dao cua A
    return [row[n:] for row in aug]
```

1.2.6 Thuật toán tìm hạng ma trận và cơ sở

Thuật toán tận dụng kết quả của phép khử Gauss để xác định hạng và trích xuất trực tiếp các vector cơ sở mà không cần phải tính toán lại từ đầu, đồng thời giải hệ phương trình thuần nhất thông qua kỹ thuật thế ngược (Back-substitution) để tìm không gian nghiệm.

Đầu vào:

- Ma trận $A \in \mathbb{R}^{m \times n}$..

Đầu ra:

- Hạng ma trận rank(A).
- Tập các vector cơ sở của không gian dòng.
- Tập các vector cơ sở của không gian cột.
- Tập các vector cơ sở của không gian nghiệm.

Mô tả các bước:

Bước 1: Gọi hàm khử Gauss cho ma trận A để thu được ma trận bậc thang U.

Bước 2: Tìm Pivot và tính hạng. Duyệt qua từng hàng của U để tìm cột chứa phần tử khác 0 đầu tiên (so sánh với sai số $\epsilon = 1e - 10$ để tránh lỗi dấu phẩy động). Lưu chỉ số các cột này vào danh sách `pivot_cols`.

$$\text{rank}(A) = \text{len}(\text{pivot_cols}) \tag{10}$$

Bước 3: Trích xuất cơ sở dòng và cột.

- **Cơ sở dòng:** Lấy rank(A) hàng đầu tiên của ma trận bậc thang U.
- **Cơ sở cột:** Trích xuất các cột từ ma trận gốc A có chỉ số nằm trong `pivot_cols`.

Bước 4: Giải tìm cơ sở không gian nghiệm (Thế ngược). Xác định danh sách các cột tự do `free_cols` (các cột không chứa pivot). Với mỗi biến tự do j:

- Khởi tạo vector nghiệm x với $x_j = 1.0$, các biến tự do khác bằng 0.
- Chạy vòng lặp ngược từ dưới lên (từ hàng pivot cuối cùng) để tính giá trị các biến cơ sở:

$$x_p = \frac{-\sum(u_{ik} \cdot x_k)}{u_{ip}}$$

(với p là cột pivot, u là phần tử của U).

- Thêm vector x thu được vào tập cơ sở nghiệm.

Cài đặt thuật toán: Dưới đây là đoạn mã nguồn hiện thực hóa thuật toán bằng Python:

Listing 6: Hàm tìm hạng và các cơ sở không gian của ma trận - $rank_and_basis.py$

```python
def rank_and_basis(self, A=None):
    if A is None:
        A = getattr(self, 'A', [])

    if not A or not A[0]:
        return 0, [], [], []

    m = len(A)      # So dong
    n = len(A[0])   # So cot

    # 1. Goi ham khu Gauss
    ref_matrix = ge.gaussian_eliminate(A, None)[0]

    # 2. Tim cac cot Pivot
    pivot_cols = []
    for i in range(len(ref_matrix)):
        for j in range(n):
            if abs(ref_matrix[i][j]) > mt.Matrix.Zero:
                pivot_cols.append(j)
                break
    # 3. Tinh hang ma tran
    rank = len(pivot_cols)

    # 4. Doi voi ma tran khong gian dong
    row_basis = [ref_matrix[i] for i in range(rank)]

    # 5. Doi voi ma tran khong gian cot
    col_basis = []
    for j in pivot_cols:
        col = [A[i][j] for i in range(m)]
```

```
        col_basis.append(col)

    # 6. Giai tim khong gian nghiem, the nguoc
    null_basis = []
    free_cols = [j for j in range(n) if j not in pivot_cols]

    for free_j in free_cols:
        x = [0.0] * n
        # Dat mot bien tu do la 1, cac bien con lai la 0
        x[free_j] = 1.0

        # Chay nguoc tu dong pivot cuoi len tren de tim nghiem
        for i in range(rank - 1, -1, -1):
            p_col = pivot_cols[i]

            sum_known = sum(ref_matrix[i][k] * x[k] for k in range(p_col + 1, n))

            # Thu nghiem tu phuong trinh R[i][p_col] * x[p_col] + sum_known = 0
            if abs(ref_matrix[i][p_col]) > mt.Matrix.Zero:
                x[p_col] = -sum_known / ref_matrix[i][p_col]
            else:
                x[p_col] = 0.0

        null_basis.append(x)

    # Tra ve hang, co so cot, co so dong, co so nghiem
    return rank, col_basis, row_basis, null_basis
```

1.2.7 Kiểm thử tính đúng đắn của hàm bằng Numpy, Scipy

Mỗi hàm có 5 test cases bao gồm test cases cơ bản đến nâng cao và edge cases. Sử dụng numpy và scipy để kiểm tra cho tính đúng đắn của các hàm, file đã được tinh chỉnh và viết lại dưới dạng Jupyter Notebook.

Đầu vào:

- List các test cases cho từng hàm bao gồm cơ bản, nâng cao và edge cases

Đầu ra:

- Ma trận đã xử lí và trạng thái đúng hoặc sai (Khử Gauss, nghịch đảo)
- Định thức của ma trận và trạng thái đúng hoặc sai (Định thức)
- Trạng thái (có nghiệm, vô nghiệm, vô số nghiệm), vector nghiệm (nếu có) và trạng thái đúng hoặc sai (Giải hệ tam giác)
- Số hạng, vector cơ sở của không gian cột, dòng, nghiệm (Tính số hạng, cơ sở không gian dòng, cột, nghiệm)

Đường link tới file kiểm thử Jupyter Notebook phần 1 chạy trên Google Colab:

2 Phần 2: Phân Rã Ma Trận và Trực Quan Hóa

2.1 Lý thuyết và Phương pháp phân rã SVD

2.1.1 Cơ sở lý thuyết của thuật toán phân rã SVD

Định nghĩa Phân rã SVD:

Phân rã giá trị đơn lẻ (Singular Value Decomposition - SVD) là một phương pháp phân tích nhân tử ma trận cơ bản và mạnh mẽ trong đại số tuyến tính. Khác với phép chéo hóa (chỉ áp dụng được cho các ma trận vuông thỏa mãn một số điều kiện nhất định), SVD có thể áp dụng để phân rã một ma trận $A \in \mathbb{R}^{m \times n}$ bất kỳ với kích thước tùy ý. Về mặt hình học, SVD cho thấy mọi phép biến đổi tuyến tính đều có thể được phân giải thành chuỗi ba phép biến đổi cơ bản: một phép quay, một phép co giãn theo các trục tọa độ, và một phép quay khác.

Công thức phân rã tổng quát:

Định lý SVD khẳng định rằng mọi ma trận A đều có thể được phân tích thành dạng tích của ba ma trận như sau:

$$A = U \Sigma V^T \tag{11}$$

Trong đó:

- $U \in \mathbb{R}^{m \times m}$ là ma trận trực giao ($U^T U = I_m$). Các cột của U được gọi là các *vector đơn lẻ trái* (left singular vectors) và chúng đóng vai trò là các vector riêng trực chuẩn của ma trận AA^T.

- $\Sigma \in \mathbb{R}^{m \times n}$ là ma trận đường chéo (chữ nhật). Các phần tử trên đường chéo chính σ_i được gọi là các *giá trị đơn lẻ* (singular values) của A. Các giá trị này luôn không âm ($\sigma_i \geq 0$) và theo quy ước luôn được sắp xếp giảm dần ($\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$).
- $V^T \in \mathbb{R}^{n \times n}$ là chuyển vị của ma trận trực giao V ($V^T V = I_n$). Các cột của V được gọi là các *vector đơn lẻ phải* (right singular vectors) và chúng là các vector riêng trực chuẩn của ma trận đối xứng $A^T A$.

2.1.2 Mối liên hệ với Bài toán Trị riêng và cách xác định:

Quá trình tính toán SVD trong thực tế thường được xây dựng thông qua việc giải bài toán trị riêng. Từ công thức SVD tổng quát, khi nhân ma trận A với chuyển vị của nó, ta có:

$$A^T A = (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T U^T U \Sigma V^T = V(\Sigma^T \Sigma)V^T \tag{12}$$

Dựa vào phương trình trên, tính chất cốt lõi để xây dựng thuật toán phân rã SVD được thể hiện qua các bước:

- Giá trị đơn lẻ σ_i được xác định bằng cách lấy căn bậc hai các trị riêng không âm λ_i của ma trận đối xứng $A^T A$: $\sigma_i = \sqrt{\lambda_i}$.
- Các vector cột v_i của ma trận V chính là các vector riêng tương ứng với trị riêng λ_i của $A^T A$. Ta có thể áp dụng các thuật toán như lặp Jacobi để tìm hệ vector riêng này.
- Các vector cột u_i của ma trận U ứng với các $\sigma_i > 0$ có thể được trích xuất nhanh chóng qua hệ thức: $u_i = \frac{1}{\sigma_i} A v_i$. Trong trường hợp A bị suy giảm hạng (có các $\sigma_i = 0$) hoặc không vuông, các cột còn thiếu của U sẽ được tìm bằng phương pháp trực giao hóa Gram-Schmidt để đảm bảo tính trực giao tổng thể của ma trận U.

2.2 Cơ sở lý thuyết về Chéo hóa ma trận

2.2.1 Định nghĩa và Điều kiện

Một ma trận vuông $\mathbf{A} \in \mathbb{R}^{n \times n}$ được gọi là **chéo hóa được** nếu tồn tại một ma trận khả nghịch $\mathbf{P}$ và một ma trận đường chéo $\mathbf{D}$ sao cho:

$$\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1} \tag{13}$$

Trong đó:

- Các phần tử trên đường chéo chính của $\mathbf{D}$ là các trị riêng $\lambda_1, \lambda_2, \ldots, \lambda_n$ của $\mathbf{A}$.
- Các cột của $\mathbf{P}$ là các vector riêng tương ứng độc lập tuyến tính.

2.2.2 Trường hợp Ma trận Đối xứng

Đối với ma trận đối xứng thực ($\mathbf{S} = \mathbf{S}^T$), theo *Định lý Phổ (Spectral Theorem)*, ma trận luôn chéo hóa trực giao được. Nghĩa là tồn tại ma trận trực giao $\mathbf{V}$ ($\mathbf{V}^T = \mathbf{V}^{-1}$) sao cho:

$$\mathbf{S} = \mathbf{V}\mathbf{D}\mathbf{V}^T \tag{14}$$

2.3 Cài đặt thuật toán SVD và Chéo hóa ma trận

2.3.1 Thuật toán phân rã SVD

Quá trình phân rã giá trị đơn lẻ (SVD) được nhóm cài đặt từ đầu qua hàm `svd(A)`. Thuật toán được thiết kế dựa trên việc giải bài toán trị riêng cho ma trận A^TA và tính toán tuần tự các ma trận U, Σ, V^T.

Đầu vào:

- Ma trận A bất kỳ, kích thước $m \times n$. Nếu ma trận rỗng, hàm sẽ ném ra lỗi `ValueError`.

Đầu ra:

- Ba ma trận: U $(m \times m)$, Σ $(m \times n)$, và V^T $(n \times n)$.

Mô tả các bước cài đặt chi tiết:

Bước 1: Tính ma trận $S = A^TA$

- Sử dụng hàm `transpose(A)` để tìm ma trận chuyển vị A^T.
- Sử dụng hàm `matmul(A_T, A)` để nhân ma trận, tạo ra ma trận vuông S kích thước $n \times n$.

Bước 2: Tìm trị riêng và vector riêng

- Gọi hàm `jacobi_eigen(S)` để tìm danh sách các trị riêng (`eigenvalues`) và ma trận chứa các vector riêng (`V_mat`).

Bước 3: Tính giá trị đơn lẻ và xây dựng Σ, V^T

- Tạo danh sách `eig_pairs` để ghép cặp từng trị riêng với vector cột riêng tương ứng.

- Sắp xếp `eig_pairs` theo thứ tự giảm dần của trị riêng.
- Duyệt qua danh sách đã sắp xếp: tính giá trị đơn lẻ $\sigma = \sqrt{\max(0, val)}$ (dùng max để tránh lỗi căn bậc hai của số âm do sai số float). Thêm σ vào mảng `singular_values` và thêm vector tương ứng làm một dòng của ma trận `V_T`.
- Tạo ma trận `V_final` bằng cách chuyển vị `V_T`.
- Khởi tạo ma trận rỗng Σ (toàn số 0) kích thước $m \times n$. Gán các giá trị trong `singular_values` vào đường chéo chính của Σ.

Bước 4: Tính các cột của ma trận U

- Khởi tạo mảng `U_cols` để lưu các cột của U và thiết lập ngưỡng sai số `tol = 1e-9`.
- Duyệt i từ 0 đến $n-1$: Nếu giá trị đơn lẻ thứ i lớn hơn `tol`, trích xuất vector cột v_i từ `V_final`. Tính Av_i thông qua hàm `matmul`, sau đó chia từng phần tử của Av_i cho giá trị đơn lẻ tương ứng để được vector cột u_i. Thêm u_i vào `U_cols`.

Bước 5: Hoàn thiện ma trận U bằng trực giao hóa Gram-Schmidt

- Trong trường hợp số cột của `U_cols` nhỏ hơn m (ma trận thiếu cột), sử dụng vòng lặp `while` để bổ sung thêm vector.
- Khởi tạo vector cơ sở tiêu chuẩn e (có phần tử thứ `basis_idx` bằng 1, còn lại bằng 0).
- Thực hiện trực giao hóa: Duyệt qua các vector u hiện có trong `U_cols`, tính tích vô hướng (dot product) giữa e và u, sau đó lấy e trừ đi $(dot \times u)$.
- Tính độ dài (norm) của vector e mới. Nếu norm $> tol$, chia từng phần tử của e cho norm để chuẩn hóa và đưa vào `U_cols`.
- Sau khi `U_cols` đủ m cột, dùng hàm `transpose` để chuyển danh sách các cột này thành ma trận U hoàn chỉnh và trả về bộ ba (U, Σ, V^T).

Cài đặt thuật toán:

Listing 7: Hàm phân rã ma trận SVD - decomposition.py

```python
import math
class SVDDecomposition:
    @staticmethod
```

```python
    def transpose(A):
        """Tinh ma tran chuyen vi"""
        return [[A[i][j] for i in range(len(A))] for j in range(len(A[0]))]

    @staticmethod
    def matmul(A, B):
        """Nhan hai ma tran"""
        m, n = len(A), len(A[0])
        p = len(B[0])
        C = [[0.0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
        return C

    @staticmethod
    def vector_norm(v):
        """Tinh chuan Euclidean (norm-2) cua vector"""
        return math.sqrt(sum(x * x for x in v))

    @staticmethod
    def jacobi_eigen(S, tol=1e-9, max_iter=1000):
        """
        Thuat toan Jacobi tim tri rieng va vector rieng cho ma tran doi xung S.
        Tra ve danh sach tri rieng (eigenvalues) va ma tran vector rieng (V).
        """
        n = len(S)
        # Khoi tao V la ma tran don vi
        V = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
        S_copy = [row[:] for row in S]

        for _ in range(max_iter):
            # Tim phan tu ngoai duong cheo co tri tuyet doi lon nhat
            max_val = 0.0
            p, q = 0, 1
```

```
        for i in range(n):
            for j in range(i + 1, n):
                if abs(S_copy[i][j]) > max_val:
                    max_val = abs(S_copy[i][j])
                    p, q = i, j

        # Neu phan tu lon nhat da du nho
        if max_val < tol:
            break

        # Tinh goc xoay theta
        tau = (S_copy[q][q] - S_copy[p][p]) / (2.0 * S_copy[p][q])
        t = math.copysign(1.0, tau) / (abs(tau) + math.sqrt(1.0 + tau * tau))
        c = 1.0 / math.sqrt(1.0 + t * t)
        s = t * c

        # Cap nhat V va S_copy
        for i in range(n):
            # V
            vip = V[i][p]
            viq = V[i][q]
            V[i][p] = c * vip - s * viq
            V[i][q] = s * vip + c * viq

            # Cap nhat cac dong trong S
            if i != p and i != q:
                sip = S_copy[i][p]
                siq = S_copy[i][q]
                S_copy[i][p] = c * sip - s * siq
                S_copy[p][i] = S_copy[i][p]
                S_copy[i][q] = s * sip + c * siq
                S_copy[q][i] = S_copy[i][q]

        # Cap nhat rieng cac phan tu p, q
        spp = S_copy[p][p]
        sqq = S_copy[q][q]
```

```
            spq = S_copy[p][q]

            S_copy[p][p] = c * c * spp - 2 * s * c * spq + s * s * sqq
            S_copy[q][q] = s * s * spp + 2 * s * c * spq + c * c * sqq
            S_copy[p][q] = 0.0
            S_copy[q][p] = 0.0

        eigenvalues = [S_copy[i][i] for i in range(n)]
        return eigenvalues, V

    @staticmethod
    def svd(A):
        """
        Phan ra SVD: A = U * Sigma * V^T
        Tra ve U, Sigma, V_T
        """
        if not A or not A[0]:
            raise ValueError("Ma tran A khong duoc rong.")

        m = len(A)
        n = len(A[0])

        tol = 1e-9
        # Tinh A_T * A
        A_T = SVDDecomposition.transpose(A)
        ATA = SVDDecomposition.matmul(A_T, A)

        # Tim tri rieng va vector rieng cua ATA
        # eigenvalues: n phan tu, V: n x n
        eigenvalues, V = SVDDecomposition.jacobi_eigen(ATA)

        # Sap xep tri rieng va vector rieng theo thu tu giam dan cua tri rieng
        eigen_pairs = []
        for i in range(len(eigenvalues)):
            # Lay tri tuyet doi de tranh loi so am cuc nho
            val = max(0, eigenvalues[i])
```

```
        # V la ma tran ma cac cot la vector rieng
        col = [V[row][i] for row in range(len(V))]
        eigen_pairs.append((val, col))

    eigen_pairs.sort(key=lambda x: x[0], reverse=True)

    sorted_eigenvalues = [p[0] for p in eigen_pairs]
    sorted_V_cols = [p[1] for p in eigen_pairs]

    # Khoi tao Sigma va gan gia tri suy bien
    Sigma = [[0.0] * n for _ in range(m)]
    singular_values = []
    for i in range(min(m, n)):
        val = math.sqrt(max(0.0, sorted_eigenvalues[i]))
        Sigma[i][i] = val
        singular_values.append(val)

    # Tinh ma tran U (m x m)
    # Cong thuc: u_i = (1 / sigma_i) * A * v_i
    U_cols = []

    for i in range(min(m, n)):
        if singular_values[i] > tol:
            # v_i la cot thu i cua V
            v_i = sorted_V_cols[i]
            # A * v_i
            u_i = [sum(A[r][c] * v_i[c] for c in range(n)) for r in range(m)]
            # Chuan hoa u_i
            norm = SVDDecomposition.vector_norm(u_i)
            U_cols.append([x / norm for x in u_i])

    # Neu U chua du m cot, bo sung bang Gram-Schmidt
    basis_idx = 0
    while len(U_cols) < m:
        # Tao vector co so tieu chuan e_i
        e = [0.0] * m
```

```
        e[basis_idx] = 1.0

        # Truc giao hoa voi cac cot da co trong U
        temp_e = e[:]
        for u in U_cols:
            dot = sum(temp_e[k] * u[k] for k in range(m))
            for k in range(m):
                temp_e[k] -= dot * u[k]

        norm = SVDDecomposition.vector_norm(temp_e)

        if norm > tol:
            # Chuan hoa
            U_cols.append([x / norm for x in temp_e])
        basis_idx += 1

    # Ma tran U hoan chinh
    U = SVDDecomposition.transpose(U_cols)
    V_T = sorted_V_cols
    return U, Sigma, V_T
```

2.3.2 Thuật toán chéo hoá ma trận

Quá trình chéo hóa ma trận được nhóm cài đặt qua hàm `diagonalize(A)`. Do chương trình sử dụng thuật toán Jacobi để tìm trị riêng, thuật toán này được thiết kế chuyên biệt để giải quyết các ma trận vuông đối xứng, cho phép tối ưu hóa quá trình tìm ma trận nghịch đảo.

Đầu vào:

- Ma trận A kích thước $n \times n$.
- Ma trận A bắt buộc phải là ma trận đối xứng ($A = A^T$). Nếu vi phạm điều kiện kích thước hoặc tính đối xứng, hàm sẽ ném ra lỗi `ValueError`.

Đầu ra:

- Ba ma trận: Ma trận làm chéo P $(n \times n)$, ma trận đường chéo D $(n \times n)$, và ma trận nghịch đảo P^{-1} $(n \times n)$.

Mô tả các bước cài đặt chi tiết:

Bước 1: Kiểm tra tính hợp lệ của đầu vào

- Kiểm tra ma trận rỗng.
- Kiểm tra tính vuông của ma trận (số hàng m phải bằng số cột n).
- Gọi hàm trợ trợ `is_symmetric(A)` để kiểm tra tính đối xứng. Hàm này duyệt qua nửa trên của ma trận và so sánh $A[i][j]$ với $A[j][i]$ dựa trên một sai số cho phép (`tol = 1e-9`).

Bước 2: Tìm trị riêng và ma trận vector riêng

- Sử dụng hàm `jacobi_eigen(A)` được kế thừa từ lớp `SVDDecomposition`. Thuật toán Jacobi sẽ trả về trực tiếp mảng các trị riêng (`eigenvalues`) và ma trận trực giao P chứa các vector riêng tương ứng làm các cột.

Bước 3: Xây dựng ma trận đường chéo D

- Khởi tạo ma trận rỗng D kích thước $n \times n$ (toàn số 0).
- Duyệt vòng lặp gán lần lượt các giá trị trong mảng `eigenvalues` vào vị trí đường chéo chính $D[i][i]$.

Bước 4: Tính ma trận nghịch đảo P^{-1}

- Theo tính chất của đại số tuyến tính, vì A là ma trận thực đối xứng, các vector riêng sinh ra từ thuật toán Jacobi tạo thành một hệ trực giao. Do đó, ma trận P là ma trận trực giao.
- Suy ra: $P^{-1} = P^T$. Thuật toán tối ưu hóa việc tìm ma trận nghịch đảo bằng cách chỉ cần gọi hàm `transpose(P)` thay vì phải dùng các thuật toán khử Gauss phức tạp.
- Trả về bộ ba kết quả (P, D, P^{-1}).

Cài đặt thuật toán:

Listing 8: Hàm chéo hóa ma trận đối xứng

```python
import decomposition as decom

class Diagonalization:
    @staticmethod
```

```python
    def is_symmetric(A, tol=1e-9):
        """Kiem tra ma tran doi xung."""
        n = len(A)
        for i in range(n):
            for j in range(i + 1, n):
                if abs(A[i][j] - A[j][i]) > tol:
                    return False
        return True

    @staticmethod
    def diagonalize(A):
        """
        Cheo hoa ma tran A: A = P * D * P^-1
        """
        if not A or not A[0]:
            raise ValueError("Ma tran A khong duoc rong.")

        m = len(A)
        n = len(A[0])

        if m != n:
            raise ValueError("Ma tran A phai la ma tran vuong de co the cheo hoa.")

        if not Diagonalization.is_symmetric(A):
            raise ValueError("Thuat toan Jacobi hien tai chi ho tro cheo hoa ma tran doi
                xung.")

        # Tim tri rieng va vector rieng bang thuat toan Jacobi tu file decomposition.py
        eigenvalues, P = decom.SVDDecomposition.jacobi_eigen(A)

        # Xay dung ma tran duong cheo D
        D = [[0.0] * n for _ in range(n)]
        for i in range(n):
            D[i][i] = eigenvalues[i]

        # Tinh P^-1
```

```
        P_inv = decom.SVDDecomposition.transpose(P)

        return P, D, P_inv

    @staticmethod
    def reconstruct(P, D, P_inv):
        """Ham ho tro kiem tra ket qua cheo hoa"""
        PD = decom.SVDDecomposition.matmul(P, D)
        A_reconstructed = decom.SVDDecomposition.matmul(PD, P_inv)
        return A_reconstructed
```

2.3.3 Kiểm thử hàm phẫn rã SVD và chéo hoá ma trận

Đầu vào:

- List các test cases cho 2 hàm phân rã SVD và chéo hoá bao gồm cơ bản, nâng cao và edge cases

Đầu ra:

- Ma trận U, $Sigma$, V^T và trạng thái đúng hoặc sai dựa vào $A \approx U \cdot \sigma \cdot V^T$ dành cho hàm phân rã SVD
- Ma trận P, D, P^{-1} vả trạng thái đúng hoặc sai dựa vào $A \approx P \cdot D \cdot P^{-1}$ dành cho hàm chéo hoá ma trận

Đường link tới file kiểm thử Jupyter Notebook phần 2 chạy trên Google Colab:

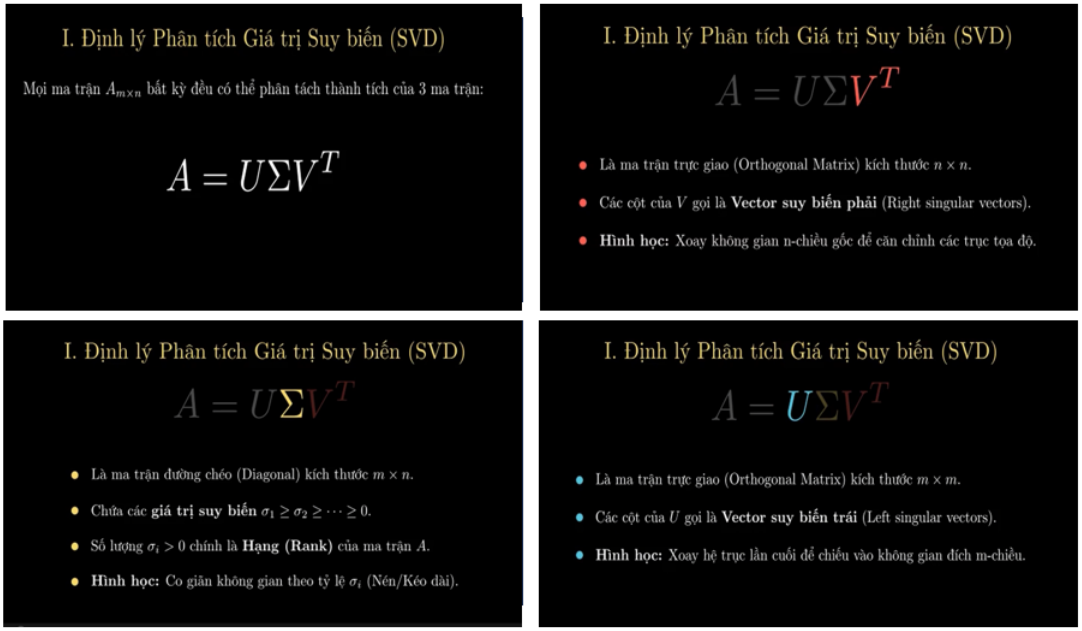
2.4 Mô tả kịch bản trực quan hóa (Manim)

Nhằm giúp người xem hiểu rõ bản chất hình học và đại số của các phép biến đổi ma trận, nhóm đã tiến hành xây dựng một kịch bản video mô phỏng trực quan sử dụng thư viện đồ họa toán học Manim. Kịch bản video được thiết kế logic, đi từ lý thuyết cơ bản đến chứng minh đại số và ứng dụng thực tiễn, bao gồm ba phần chính:

2.4.1 Trình bày lý thuyết và mô phỏng phân rã SVD

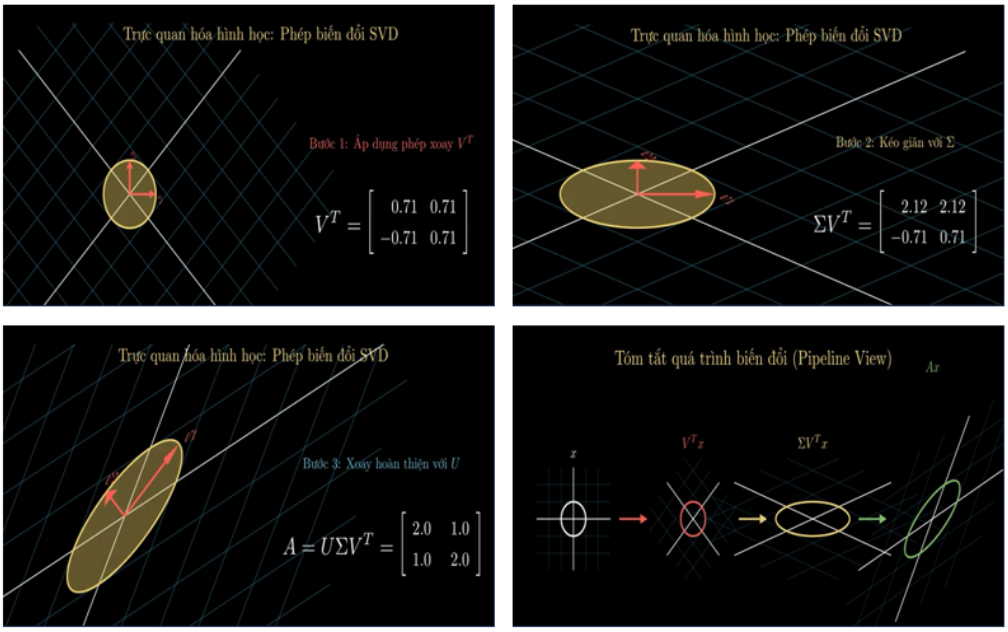
- **Mục tiêu:** Trình bày lý thuyết SVD và minh họa bản chất hình học của phép phân rã.

- **Mô tả kịch bản:**
  - *Mở đầu:* Video bắt đầu bằng việc giới thiệu định lý SVD với công thức tổng quát $A = U\Sigma V^T$.
  - *Giải thích bản chất:* Kịch bản chia phép biến đổi không gian thành 3 bước độc lập: Xoay hệ trục tọa độ (V^T), Co giãn không gian (Σ), và Xoay chiếu về đích (U).



Hình 1: Ảnh minh họa cho phần trình bày lý thuyết SVD.

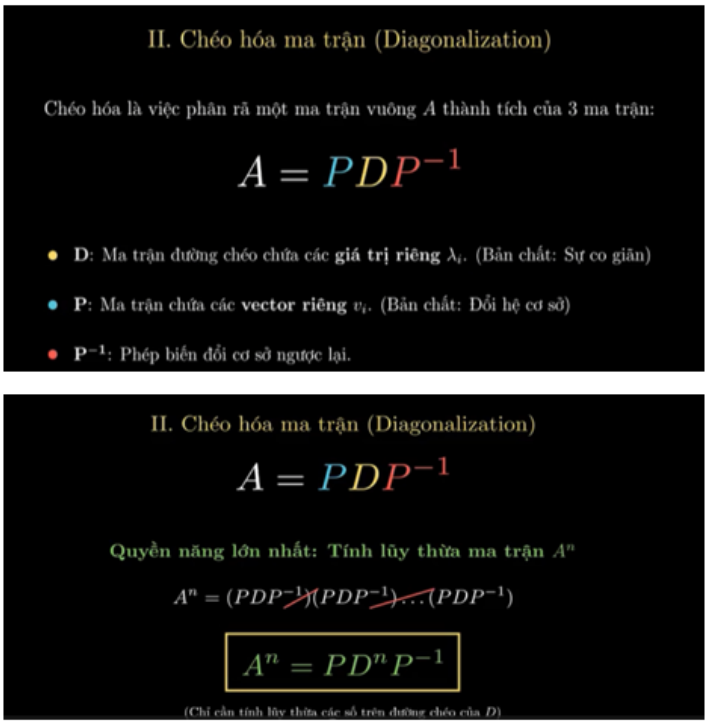
- **Mô phỏng trực quan:** Nhóm sử dụng ma trận thực tế $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ tác động lên một đường tròn đơn vị và hai vector cơ sở trong không gian 2D. Các hiệu ứng hình học (apply_matrix) được áp dụng tuần tự theo từng ma trận thành phần, giúp người xem thấy rõ quá trình đường tròn bị xoay, bóp méo thành hình elip và định vị lại.



Hình 2: Mô phỏng trực quan SVD trong không gian 2D.

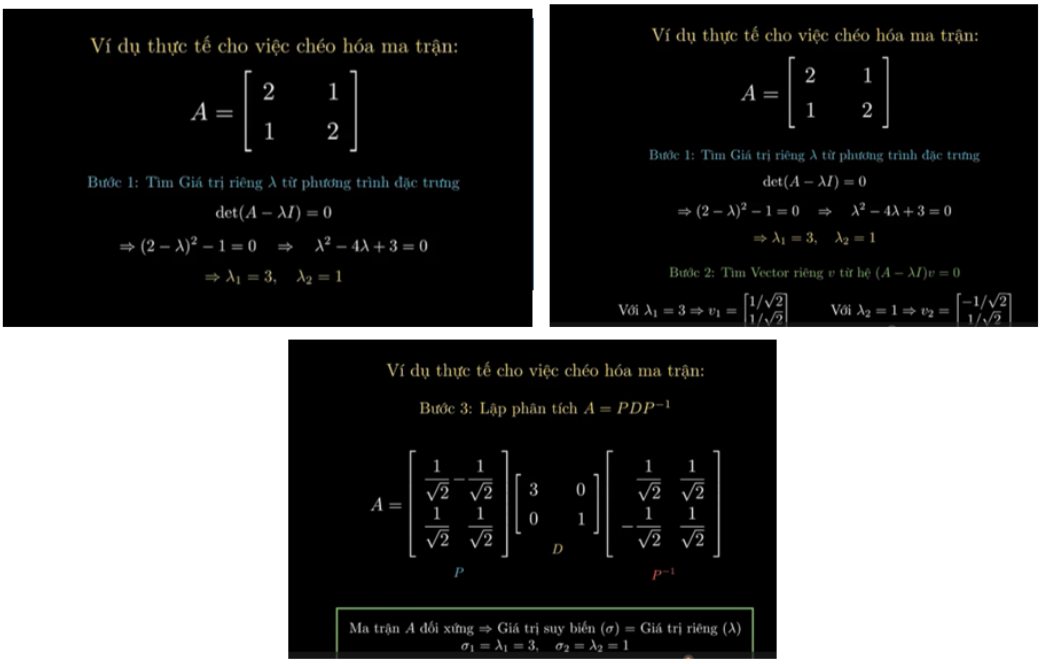
2.4.2 Trình bày lý thuyết và mô phỏng Chéo hóa ma trận

- **Mục tiêu:** Trình bày lý thuyết chéo hóa và ứng dụng trong tính toán lũy thừa ma trận.
- **Mô tả kịch bản:**
  - Giới thiệu công thức chéo hóa $A = PDP^{-1}$.
  - Nhóm lồng ghép hiệu ứng triệt tiêu đại số để làm nổi bật sức mạnh của phép chéo hóa trong việc tính lũy thừa ma trận: $A^n = PD^nP^{-1}$.



Hình 3: Ảnh minh họa cho lý thuyết chéo hóa.

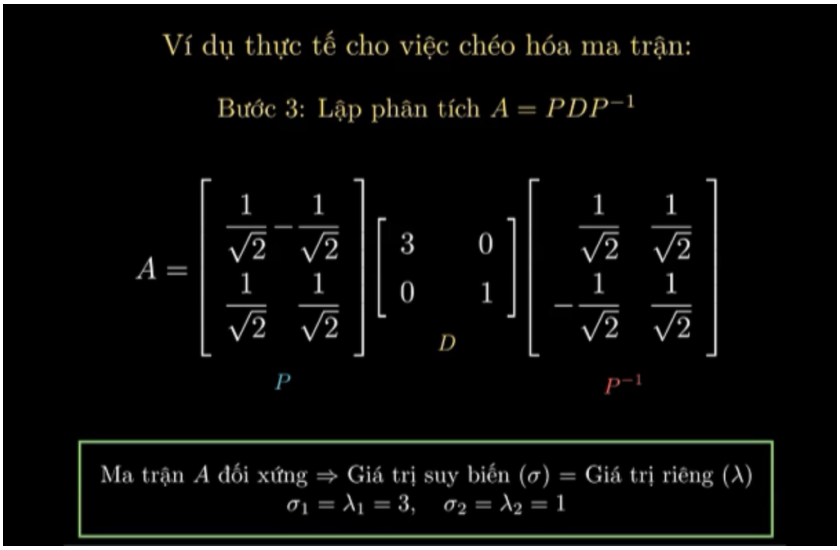
- **Mô phỏng trực quan:** Minh họa từng bước giải bài toán chéo hóa cho ma trận đối xứng. Giao diện video mô phỏng lại quá trình giải phương trình đặc trưng để tìm trị riêng λ và vector riêng v. Điểm nhấn là hiệu ứng "lắp ráp": các trị riêng di chuyển để tạo thành ma trận đường chéo D, trong khi các vector riêng ghép lại thành các cột của ma trận P.



Hình 4: Ví dụ minh họa quá trình lắp ráp ma trận chéo hóa.

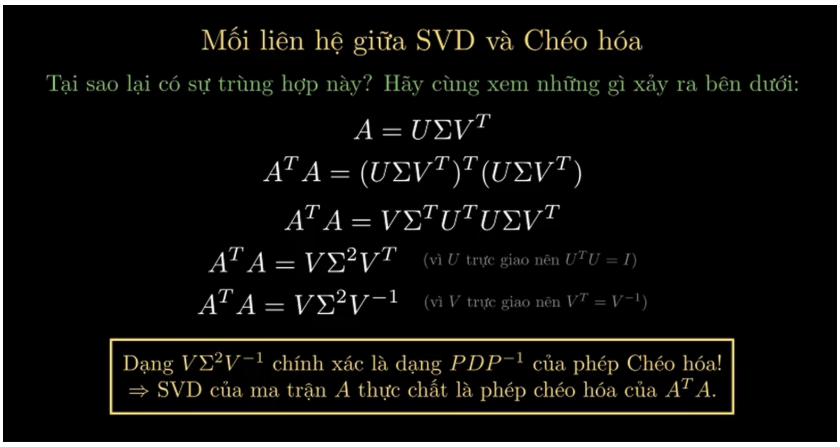
2.4.3 Mối liên hệ đại số và Ứng dụng nén ảnh

- **Mục tiêu:** Chứng minh mối liên hệ chặt chẽ giữa hai phương pháp và minh họa tính ứng dụng của SVD trong thực tế.



Hình 5: Tổng quan mối liên hệ giữa SVD và Chéo hóa.

- **Chứng minh mối liên hệ:** Kịch bản sử dụng hiệu ứng biến đổi công thức (TransformMatchingTex) để đi từ công thức $A^TA = (U\Sigma V^T)^T(U\Sigma V^T)$. Thông qua việc triệt tiêu ma trận trực giao $U^TU = I$ và đổi $V^T = V^{-1}$, hệ thống suy ra kết quả $A^TA = V\Sigma^2V^{-1}$. Phân cảnh này đưa ra lời giải thích sắc bén: Phép SVD của ma trận A thực chất chính là phép chéo hóa của ma trận A^TA, với trị riêng λ_i bằng bình phương của giá trị suy biến σ_i.



Hình 6: Phân cảnh chứng minh đại số trên Manim.

- **Ứng dụng nén hình ảnh:** Phân cảnh cuối chuyển sang mô phỏng thuật toán SVD rút gọn (Truncated SVD) trên một hình ảnh thực tế (được xem như một ma trận điểm ảnh).

Video hiển thị một thanh trượt tham số k (số lượng giá trị suy biến được giữ lại). Khi k tăng dần, bức ảnh từ dạng các mảng mờ nhạt dần trở nên sắc nét, qua đó trực quan hóa khả năng loại bỏ dữ liệu dư thừa và tối ưu hóa không gian lưu trữ của thuật toán SVD.

Hình 7: Mô phỏng ứng dụng nén ảnh bằng Truncated SVD.

3 Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

3.1 Cơ sở lý thuyết về sai số và tính ổn định số

3.1.1 Chuẩn và Số điều kiện

Trong tính toán số học, khả năng giải quyết chính xác hệ $Ax = b$ phụ thuộc rất lớn vào số điều kiện $\kappa(A)$. Sử dụng phân rã SVD, ta có:

$$\kappa(A)_2 = \frac{\sigma_{max}}{\sigma_{min}} \tag{15}$$

Một bài toán có $\kappa(A)$ lớn được gọi là bài toán "xấu" (ill-conditioned), nơi mà sai số làm tròn có thể phá hủy hoàn toàn ý nghĩa của kết quả tính toán.

3.1.2 Sai số tương đối

Để đánh giá kết quả thực nghiệm, chúng tôi sử dụng chuẩn bậc 2 của vector dư (residual vector) chia cho chuẩn của vector vế phải b:

$$Error = \frac{\|A\hat{x} - b\|_2}{\|b\|_2} \tag{16}$$

3.1.3 So sánh các phương pháp được thực hiện

Tiêu chí	**Khử Gauss**	**Gauss-Seidel**	**Phân rã SVD**
Độ ổn định	Trung bình	Phụ thuộc hội tụ	Rất cao
Ưu điểm	Phổ biến, trực tiếp	Tiết kiệm bộ nhớ	Giải được mọi hệ
Nhược điểm	Nhạy với sai số	Khó hội tụ với ma trận xấu	Tính toán phức tạp

Bảng 4: Bảng tổng hợp so sánh các phương pháp giải hệ phương trình

3.1.4 Phương pháp lặp Gauss-Seidel

Phương pháp Gauss-Seidel là một thuật toán lặp được sử dụng để giải hệ phương trình tuyến tính $Ax = b$ với A là ma trận vuông cấp n. Đây là một cải tiến của phương pháp lặp Jacobi, trong đó các giá trị mới của biến được sử dụng ngay lập tức sau khi chúng được tính toán thay vì đợi đến bước lặp kế tiếp.

Cơ sở toán học

Xét hệ phương trình tuyến tính dưới dạng ma trận $Ax = b$. Ta phân rã ma trận A thành tổng của ba ma trận:

$$A = D + L + U \tag{17}$$

Trong đó:

- $D = \text{diag}(a_{11}, a_{22}, \ldots, a_{nn})$ là ma trận đường chéo.
- L là ma trận tam giác dưới nghiêm ngặt (các phần tử phía trên và trên đường chéo bằng 0).
- U là ma trận tam giác trên nghiêm ngặt (các phần tử phía dưới và trên đường chéo bằng 0).

Hệ phương trình có thể viết lại dưới dạng:

$$(D+L)x + Ux = b \tag{18}$$

$$(D+L)x = b - Ux \tag{19}$$

Từ đó, ta thiết lập công thức truy hồi cho bước lặp thứ $(k+1)$ như sau:

$$x^{(k+1)} = (D+L)^{-1}(b - Ux^{(k)}) \tag{20}$$

Dưới dạng khai triển cho từng thành phần x_i, công thức lặp Gauss-Seidel được mô tả bởi:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \ldots, n \tag{21}$$

Điều kiện hội tụ

Một điều kiện đủ để phương pháp Gauss-Seidel hội tụ với mọi vector xấp xỉ ban đầu $x^{(0)}$ là ma trận A phải là ma trận **chéo trội nghiêm ngặt** (Strictly Diagonally Dominant):

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i = 1, 2, \ldots, n \tag{22}$$

Ngoài ra, thuật toán cũng hội tụ nếu A là ma trận đối xứng xác định dương (Symmetric Positive Definite - SPD).

Tiêu chuẩn dừng

Quá trình lặp sẽ dừng lại khi nghiệm xấp xỉ đạt được độ chính xác ϵ cho trước. Tiêu chuẩn dừng thường được xác định dựa trên chuẩn Euclidean của hiệu hai bước lặp liên tiếp hoặc chuẩn của số dư:

- Sai số giữa hai bước lặp: $\|x^{(k+1)} - x^{(k)}\|_2 < \epsilon$
- Chuẩn số dư: $\|Ax^{(k+1)} - b\|_2 < \epsilon$

3.2 Mô tả thuật toán và cài đặt

3.2.1 Các bước thực hiện thuật toán Gauss-Seidel

- **Bước 1:** Khởi tạo và thiết lập vector nghiệm ban đầu $x^{(0)} = \mathbf{0}$, ngưỡng sai số ϵ (tolerance) và số lần lặp tối đa N_{max}.

- **Bước 2:** Với mỗi bước lặp $k = 0, 1, 2, \ldots, N_{max} - 1$:
  - Lưu trữ nghiệm cũ: $x_{old} = x^{(k)}$.
  - Cập nhật từng thành phần x_i của vector nghiệm ($i = 1, \ldots, n$):

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right) \tag{23}$$

  - Kiểm tra tính hợp lệ: Nếu $|a_{ii}| \approx 0$, dừng thuật toán và thông báo lỗi.

- **Bước 3:** Kiểm tra hội tụ bằng cách tính chuẩn Euclidean của hiệu hai bước lặp:

$$\text{diff} = \sqrt{\sum_{i=1}^{n} (x_i^{(k+1)} - x_i^{(k)})^2} \tag{24}$$

  Nếu diff $< \epsilon$, thuật toán dừng và trả về nghiệm $x^{(k+1)}$.

- **Bước 4:** Nếu đạt đến N_{max} mà không thỏa mãn điều kiện dừng, trả về kết quả hiện tại.

3.2.2 Cài đặt thuật toán Gauss-Seidel

Listing 9: Hàm giải thuật toán Gauss-Seidel tách từ solvers.py

```python
import math
import part1.matrix as mt
import part1.gaussian_eliminate as ge
import part1.back_substitution as bs
import part2.decomposition as dcp

class Solvers:
    def __init__(self, matrix_obj):
        # Khoi tao voi mot doi tuong Matrix
        self.A = matrix_obj.A
        self.b = matrix_obj.b
        self.n = matrix_obj.n
        self.m = matrix_obj.m
        self.Zero = matrix_obj.Zero

```

```python
    def solve_gauss_seidel(self, tol=1e-6, max_iter=10000):
        """Giai Ax = b bang phuong phap lap Gauss-Seidel"""
        # Kiem tra ma tran co vuong khong truoc khi thuc hien
        if self.n != self.m:
            raise ValueError("Gauss-Seidel chi ap dung cho ma tran vuong.")

        # Khoi tao x la vector 0
        x = [0.0] * self.n

        # Kiem tra dieu kien cheo troi chat
        if not self.check_diagonal_dominance(self.A):
            print("Ma tran khong cheo troi  Gauss-Seidel co the khong hoi tu.")

        # Ma tran U la ma tran tam giac tren (khong bao gom duong cheo) chua cac phan tu cua A
        U = []
        for i in range(self.n):
            row = [0.0] * self.n
            for j in range(i + 1, self.n):
                row[j] = self.A[i][j]
            U.append(row)

        for k in range(max_iter):
            # Tao ban sao cua x de so sanh hoi tu
            x_old = x[:]

            for i in range(self.n):
                # Tinh sum_new: cac phan tu da duoc cap nhat o buoc lap hien tai (j < i)
                sum_new = sum(self.A[i][j] * x[j] for j in range(i))

                # Tinh sum_old: cac phan tu tu buoc lap truoc (j > i)
                sum_old = sum(self.A[i][j] * x_old[j] for j in range(i + 1, self.n))

                # Cap nhat x[i]
                # Neu tren duong cheo co phan tu bang 0, khong the thuc hien bang phuong phap nay
```

```
            if abs(self.A[i][i]) <= self.Zero:
                raise ValueError(f"Phan tu duong cheo A[{i}][{i}] bang 0.")
            x[i] = (self.b[i] - sum_new - sum_old) / self.A[i][i]

        # Kiem tra dieu kien hoi tu bang chuan Euclidean
        diff_sq_sum = sum((x[i] - x_old[i])**2 for i in range(self.n))
        # Neu khoang cach nay nho hon sai so cho phep, cac gia tri cua x da on dinh
        if math.sqrt(diff_sq_sum) < tol:
            return U, x, k + 1

    # Nghiem chua chac da chinh xac, can kiem tra bang phuong phap khac
    return U, x, max_iter
```

3.3 Mô tả quá trình kiểm nghiệm

3.3.1 Phân loại các trường hợp kiểm thử

- **Loại 1 - Ma trận chéo trội nghiêm ngặt (SDD):** Các phần tử đường chéo được thiết lập sao cho $|a_{ii}| > \sum_{j \neq i} |a_{ij}| + \text{rand}(2, 10)$. Đây là điều kiện lý tưởng cho phương pháp Gauss-Seidel.

- **Loại 2 - Ma trận chéo trội biên độ lớn:** Tương tự Loại 1 nhưng với biên độ giá trị ngẫu nhiên lớn hơn nhằm kiểm tra độ ổn định số học khi giá trị các phần tử tăng cao.

- **Loại 3 - Ma trận không chéo trội (Non-SDD):** Các phần tử đường chéo được cố ý làm nhỏ hơn nhiều so với tổng các phần tử còn lại trên hàng ($|a_{ii}| \approx 0.1 \times$ off-diag sum). Trường hợp này dùng để kiểm chứng sự phân kỳ hoặc lỗi hội tụ của phương pháp lặp.

3.3.2 Các chỉ số đo đạc

- **Thời gian thực thi (Execution Time):** Sử dụng hàm **time.perf_counter()** để đo độ trễ tính toán với độ chính xác cao.

- **Sai số thực tế (L2 Residual Error):** Được tính theo chuẩn Euclidean của vector dư:

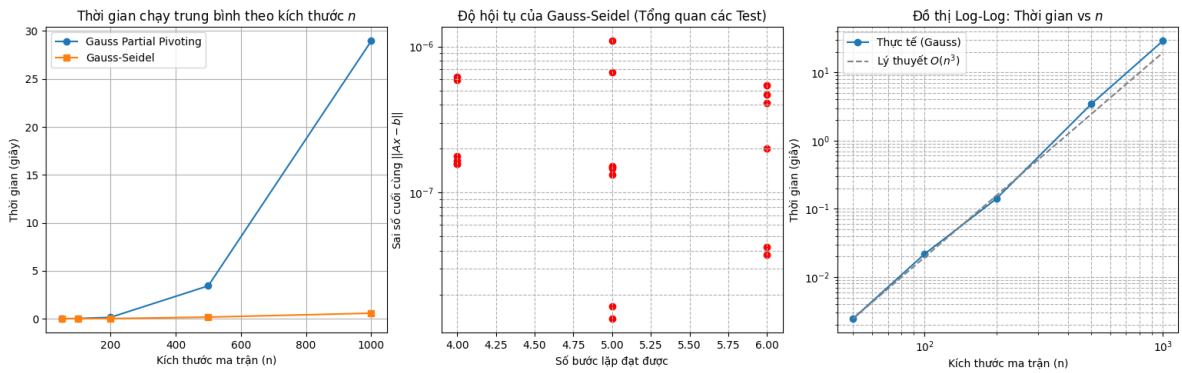
$$\text{Error} = |Ax - b|2 = \sqrt{\sum i = 1^n \left(\sum_{j=1}^{n} a_{ij} x_j - b_i \right)^2} \tag{25}$$

- **Số bước tính toán (Steps/Iterations):** Đối với phương pháp trực tiếp (Gauss, SVD), số bước mặc định là 1. Đối với Gauss-Seidel, đây là số vòng lặp cần thiết để đạt ngưỡng hội tụ $\epsilon = 10^{-6}$.

3.3.3 Quá trình kiểm thử

1. Khởi tạo ma trận A và vector b theo loại và kích thước định trước.
2. Đóng gói dữ liệu vào đối tượng `Matrix` và khởi tạo lớp `Solvers`.
3. Gọi lần lượt các hàm:
   - **solve_gauss()**: Giải bằng khử Gauss chọn phần tử trụ.
   - **solve_svd()**: Giải bằng phân rã giá trị đơn lẻ.
   - **solve_gauss_seidel()**: Giải bằng phương pháp lặp với giới hạn 1000 vòng lặp.
4. Tính toán sai số, in báo cáo so sánh kết quả trực tiếp trên màn hình console và lưu dữ liệu vào file Excel.

Biểu đồ thời gian chạy, độ hội tụ và log-log:



Hình 8: Biểu đồ thời gian chạy, độ hội tụ và log-log.

3.4 Phân tích tính ổn định số

3.4.1 Ma trận Hilbert

Định nghĩa: Ma trận Hilbert là một ma trận vuông mà các phần tử của nó được xác định bởi các đơn phân số. Đây là một ví dụ điển hình trong giải tích số về loại ma trận cực kỳ nhạy cảm với sai số.

Một ma trận Hilbert cấp n, ký hiệu là H_n, có các phần tử h_{ij} được xác định bởi công thức:

$$h_{ij} = \frac{1}{i+j-1}, \quad 1 \leq i, j \leq n \tag{26}$$

Ví dụ, ma trận Hilbert cấp 3 (H_3) có dạng:

$$H_3 = \begin{bmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{bmatrix} \tag{27}$$

Đặc điểm chính:

- H_n luôn là ma trận đối xứng và xác định dương.
- Số điều kiện $\kappa(H_n)$ tăng theo hàm mũ $e^{3.5n}$, khiến việc nghịch đảo ma trận này bằng các phương pháp trực tiếp như khử Gauss gặp sai số rất lớn khi n tăng.

3.4.2 Ma trận đối xứng xác định dương (SPD)

Ma trận SPD (Symmetric Positive Definite) đóng vai trò quan trọng trong các bài toán tối ưu hóa và vật lý kỹ thuật do tính chất hội tụ tốt của các thuật toán liên quan.

Định nghĩa: Một ma trận $A \in \mathbb{R}^{n \times n}$ được gọi là ma trận đối xứng xác định dương nếu thỏa mãn hai điều kiện:

1. **Tính đối xứng:** $A = A^T$.
2. **Tính xác định dương:** Với mọi vector $x \in \mathbb{R}^n$ khác không ($x \neq 0$), ta luôn có:

$$x^T A x > 0 \tag{28}$$

Các tính chất quan trọng:

- Tất cả các trị riêng λ_i của ma trận A đều là các số thực dương ($\lambda_i > 0$).
- Ma trận A luôn khả nghịch và ma trận nghịch đảo A^{-1} cũng là một ma trận SPD.
- Có thể phân rã ma trận A bằng phương pháp Cholesky ($A = LL^T$), đây là phương pháp giải hệ phương trình tuyến tính cực kỳ ổn định về mặt số học.

3.4.3 Mô tả phương pháp ổn định số học

Để đánh giá độ ổn định số (Numerical Stability) của một ma trận, phương pháp phân rã giá trị suy biến (Singular Value Decomposition - SVD) được sử dụng để tính toán số điều kiện (Condition Number).Số điều kiện của ma trận A, ký hiệu là $\kappa(A)$, được xác định thông qua tỷ lệ giữa giá trị suy biến lớn nhất và nhỏ nhất thu được từ phân rã SVD:

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}} \tag{29}$$

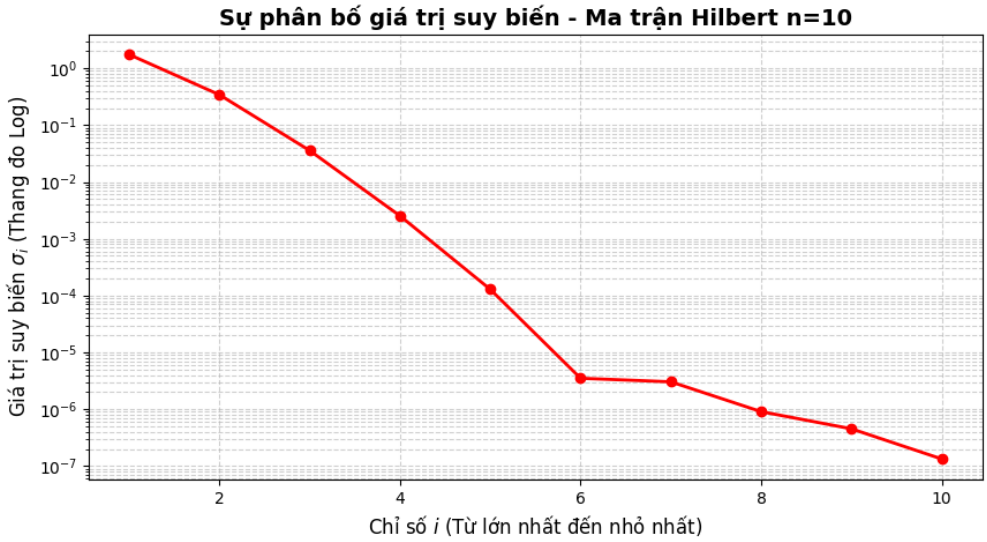
Ma trận Hilbert được chọn làm đại diện tiêu biểu cho nhóm ma trận Ill-conditioned.

- **Đặc điểm trị suy biến:** Kết quả phân tích cho thấy các giá trị suy biến σ_i giảm cực kỳ nhanh. Giá trị $\sigma_{\min}$ tiến rất gần về 0 ngay cả khi kích thước ma trận n còn nhỏ.
- **Số điều kiện:** Do $\sigma_{\min} \to 0$, số điều kiện $\kappa(H_n)$ tăng theo hàm mũ so với n. Điều này khiến việc tính toán nghịch đảo hoặc giải hệ phương trình với ma trận Hilbert gặp sai số tích lũy rất lớn.
- **Đặc điểm trị suy biến:** Đồ thị biểu diễn các giá trị suy biến gần như tạo thành một đường nằm ngang. Sự chênh lệch giữa $\sigma_{\max}$ và $\sigma_{\min}$ là không đáng kể.
- **Số điều kiện:** Số điều kiện κ của ma trận này thường rất nhỏ, dao động quanh mức $1.x$.
- **Tính ổn định:** Do $\kappa \approx 1$, thuật toán khi xử lý loại ma trận này sẽ hội tụ nhanh và giữ được độ chính xác cao bất chấp các sai số nhiễu thông thường.

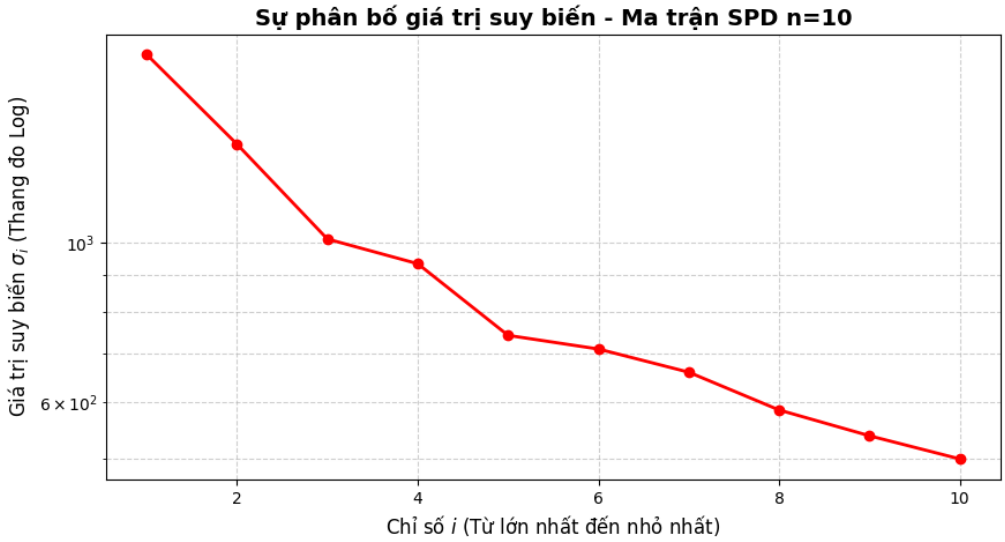
Kết luận:

- Độ tin cậy của một thuật toán (như Gauss-Seidel) không chỉ phụ thuộc vào bản thân phương pháp lặp, mà còn phụ thuộc rất lớn vào đặc tính nội tại (số điều kiện) của ma trận đầu vào.
- Phân rã SVD là công cụ mạnh mẽ nhất để "chẩn đoán" sức khỏe của một hệ phương trình tuyến tính trước khi tiến hành giải thuật.

Biểu đồ sự phân bổ giá trị suy biến:



Hình 9: Biểu đồ sự phân bổ giá trị suy biến - Ma trận Hilbert n = 10.



Hình 10: Biểu đồ sự phân bổ giá trị suy biến - Ma trận SPD n = 10.

4 Phần 4: Kết luận

4.1 Những gì đạt được thông qua đồ án

Qua quá trình tìm hiểu và cài đặt các thuật toán, nhóm đã có được sự hiểu biết sâu hơn về ma trận và các ứng dụng công nghệ của ma trận vào đời sống . Cụ thể hơn, nhóm đã làm được những điều sau:

- Sử dụng Github để sắp xếp, quản lý dự án.
- Rèn luyện khả năng lên ké hoạch và làm việc theo nhóm.
- Hiểu được cách giải hệ phương trình bằng ma trận.
- Biết cách tái sử dụng và kết hợp các hàm riêng lẻ.
- Biết cách kiểm tra và thực nghiệm cho các hàm.
- Tìm hiểu, ứng dụng Manim và Latex để xây dựng video học thuật trực quan về toán học.
- Phân tích số liệu và vẽ biểu đồ bằng Matplotlib

4.2 Những khó khăn gặp phải

Việc tìm hiểu của nhóm gặp nhiều khó khăn trong lúc tìm kiếm tài liệu tham khảo và kết hợp các file lại với nhau, nhóm cũng gặp khá nhiều vấn đề trong quá trình kiểm thử và xây dựng video Manim, cụ thể như sau:

- Các tài liệu đa số là tiếng Anh khiến việc tiếp thu và áp dụng kiến thức gặp nhiều thiếu sót.
- Việc kết hợp các hàm có sai sót trong dữ liệu đầu ra khiến nhóm phải định nghĩa lại dữ liệu đầu ra.
- Trong quá trình kiểm thử, nhóm đã gặp các edge cases với sai số nhỏ, điều đó khiến nhóm phải tái cấu trúc code để đưa ra sai số ít hơn 10^{-9}.
- Việc mô phòng video Manim gặp khó khăn khi phải tìm animation phù hợp với quá trình mô phỏng phân rã và chéo hoá ma trận.
- Thời gian và sai số của các thuật toán khử với từng loại ma trận là khác nhau nên việc đánh giá trường hợp nào sẽ sử dụng thuật khử nào trở nên khó khăn và đa dạng hơn.

4.3 Bài học rút ra

Thông qua đồ án cùng với những khó khăn mà nhóm gặp phải, nhốm đã đúc kết vả rút ra được các bài học rằng:

- Việc lên kế hoạch, định nghĩa cấu trúc code trước khi làm dự án là vô cùng quan trọng.
- Phải tìm hiểu từ nhiều tài liệu từ nghiều nguồn khác nhau
- Quản lý code bằng các message khi commit trên github là cần thiết để quản lý tiến trình dự án
- Làm việc nhóm phải có sự liên kết, giao tiếp giữa các thành viên trong nhóm để quá trình làm việc trở nên suông sẻ.

Tài liệu

[1] Gilbert Strang. *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press, 2023.

[2] Lloyd N. Trefethen & David Bau III. *Numerical Linear Algebra*. SIAM, 1997.

[3] **Gaussian Elimination to Solve Linear Equations | GeeksforGeeks**

[4] **Gaussian Elimination with Partial Pivoting (Visualization) | Terry D. Johnson (Massachusetts Institute of Technology)**

[5] **How to Calculate Matrix Inverse in Python | Jake @Scicoding**

[6] **Column Space Basis | University of Sydney**

[7] **Row Space, Column Space, and Nullspace | East Tennessee State University**

[8] **Post Jupiter Notebooks to a GitHub Repository | Google Colab**

[9] **Lecture 47 — Singular Value Decomposition | Stanford University**

[10] **Singular Value Decomposition | Machine learning cơ bản**

[11] **Singular Value Decomposition - Definition and Application**

[12] **Jacobi methods | Stanford University**

[13] **Matrix Diagonalization | GeeksforGeeks**

[14] Manim Community Developers. *Manim — Mathematical Animation Framework*, v0.18. 2024. https://docs.manim.community

[15] **Gauss Seidel Method | GeeksforGeeks**

[16] **Matplotlib: Visualization with Python**

[17] **Matplotlib.pyplot.loglog() function in Python**

[18] **Hilbert Matrix**

[19] **Hilbert Matrix Visualization**

[20] **Definition of SPD Matrix**

[21] **Symmetric Positive Definite Matrix - Basic Theorems**